



Buenos
Aires
Ciudad



Buenos Aires
aprende
Asociación de Instituciones para el Futuro



FRONT-END JS

Clase n° 12 | DOM y eventos

¡Les damos la bienvenida!



 Vamos a comenzar a grabar la clase.

Índice

1

12. DOM y eventos

- Manipulación del DOM
- Definición, alcance y su importancia para operar sobre elementos HTML
- Eventos en JS
- Eventos: ¿Que son, para qué sirven y cuáles son los más comunes?
- Escuchar un evento sobre el DOM

2

13. LocalStorage, sessionStorage y Carrito de Compras

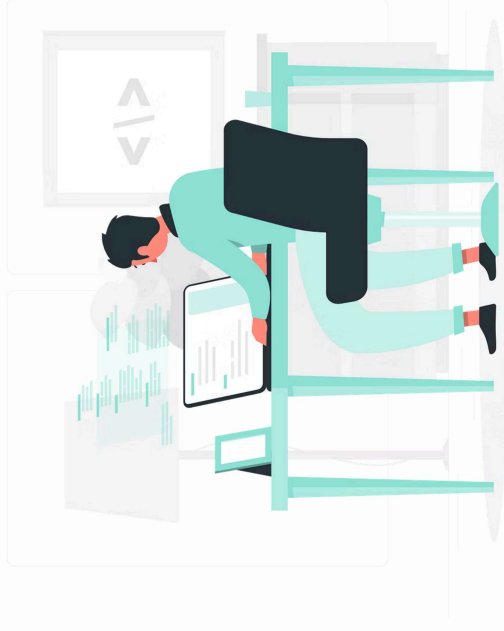
- Introducción a LocalStorage y sessionStorage
- Diferencias entre LocalStorage y sessionStorage
- Implementación de un carrito de compras utilizando LocalStorage o sessionStorage

Manipulación del DOM

DOM

¿Qué es el DOM?

El DOM es la representación en forma de árbol de una página web. Permite acceder y modificar elementos de la página dinámicamente.



Estructura Jerárquica del DOM

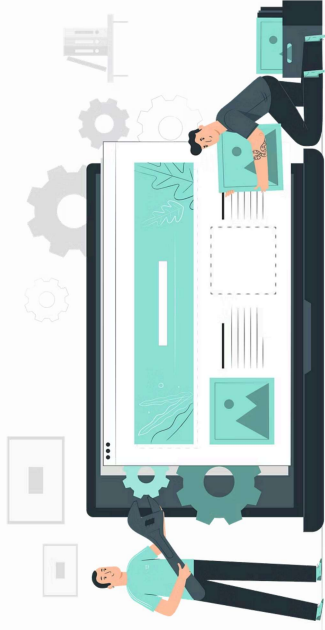


El DOM está organizado de forma jerárquica, como un árbol genealógico, en el que los elementos HTML están relacionados entre sí como una familia, siguiendo una línea de herencia. El documento HTML comienza con la etiqueta `<html>`, que tiene como "hijos/hijas" a `<head>` y `<body>`. Dentro de estos, encontramos más elementos anidados

Ejemplo:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Título de la página</title>
  </head>
  <body>
    <h1>Encabezado</h1>
    <p>Este es un párrafo.</p>
  </body>
</html>
```

Importancia del DOM en Desarrollo Web



1 Interactividad

Permite crear páginas web dinámicas y receptivas.

2 Manipulación en tiempo real

Modifica el contenido sin recargar la página.

¿Cómo acceder al DOM desde JavaScript?

- Se utiliza el objeto document, que representa el documento HTML cargado en el navegador.
- Desde este objeto, se pueden usar métodos para buscar, seleccionar y modificar elementos de la página.

Accediendo al DOM desde JavaScript

Métodos más utilizados para seleccionar y modificar elementos del DOM.

getElementById()

Selecciona un elemento según su atributo id. El id debe ser único en el documento.

```
<h1 id="titulo">Bienvenido a mi sitio</h1>
<script>
  let elemento = document.getElementById("titulo");
  console.log(elemento); // Devuelve el elemento <h1 id="titulo">...</h1>
</script>
```

Accediendo al DOM desde JavaScript

getElementsByClassName()

Devuelve una colección de elementos que comparten la misma clase.

```
<div class="caja">Caja 1</div>
<div class="caja">Caja 2</div>
<div class="caja">Caja 3</div>
<script>
  let cajas = document.getElementsByClassName("caja");
  console.log(cajas); // Devuelve una colección con las tres <div class="caja">...
</div>
// Podemos iterar sobre ellos:
for (let i = 0; i < cajas.length; i++) {
  cajas[i].style.color = "red"; // Cambiamos el color de texto a rojo
}
</script>
```

Accediendo al DOM desde JavaScript

querySelector()

Permite seleccionar el primer elemento que coincida con un selector CSS

```
<div class="contenedor">
  <p class="texto">Texto 1</p>
  <p class="texto">Texto 2</p>
</div>
<script>
  let primerParrafo = document.querySelector(".contenedor .texto");
  console.log(primerParrafo); // Devuelve el primer <p class="texto">Texto 1</p>
  // Podemos cambiar su contenido:
  primerParrafo.textContent = "Nuevo texto";
</script>
```

Accediendo al DOM desde JavaScript

`querySelectorAll()`

Devuelve todos los elementos que coinciden con el selector en forma de **NodeList** (lista de nodos)

```
<div class="contenedor">
  <p class="texto">Texto 1</p>
  <p class="texto">Texto 2</p>
</div>
<script>
```

```
let parrafos = document.querySelectorAll(".contenedor .texto");
console.log(parrafos); // Devuelve una NodeList con los dos <p class="texto">...

</p>

// Cambiamos el color de todos los párrafos seleccionados:
parrafos.forEach(parrafo => {
  parrafo.style.color = "blue";
});
</script>
```

Creación de Elementos:

- El DOM permite no solo modificar elementos existentes, sino también agregar nuevos en tiempo real. Para ello, se usa `createElement()`, que crea un nuevo nodo en memoria.
- Este nodo se añade a la página mediante `appendChild()`, integrándolo en la estructura del DOM.

Ejemplo:

```
<ul id="lista">
  <li>Elemento 1</li>
  <li>Elemento 2</li>
```

```
</ul>
<script>
  let nuevoElemento = document.createElement("li");
  nuevoElemento.textContent = "Elemento 3";
  document.getElementById("lista").appendChild(nuevoElemento);
</script>
```

Modificar Elementos

innerHTML

Te permite modificar o insertar contenido HTML dentro de un elemento. Útil cuando quieras agregar contenido más complejo, como listas, tablas o incluso secciones enteras con varias etiquetas HTML.

```
<div id="contenido">
  <p>Texto inicial</p>
</div>
<script>
  let div = document.getElementById("contenido");
  div.innerHTML = "<h2>Nuevo título</h2><p>Nuevo párrafo añadido</p>";
</script>
```

textContent:

- Solo modifica el texto dentro de un elemento, sin interpretar etiquetas HTML. Más seguro si solo estás manejando texto y querés evitar problemas de seguridad.

```
<div id="contenido">  
  <p>Texto inicial</p>  
</div>  
<script>  
  let div = document.getElementById("contenido");  
  div.textContent = "Este es solo texto, sin HTML";  
</script>
```

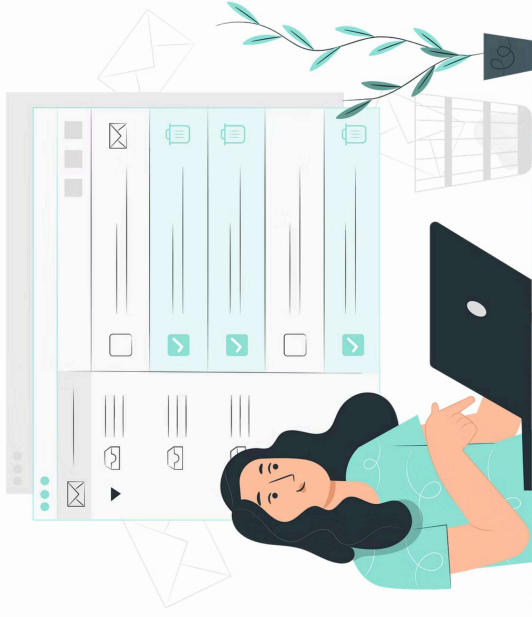
Eliminar Elementos

A veces no sólo queremos agregar o modificar elementos, sino que también necesitamos eliminarlos por completo.

Usamos el método `remove()`, que elimina un nodo del DOM.

Ejemplo:

```
<ul>
  <li id="item1">Elemento 1</li>
  <li id="item2">Elemento 2</li>
  <li id="item3">Elemento 3</li>
</ul>
<script>
  let item = document.getElementById("item2");
  item.remove();
</script>
```



Eventos en JavaScript

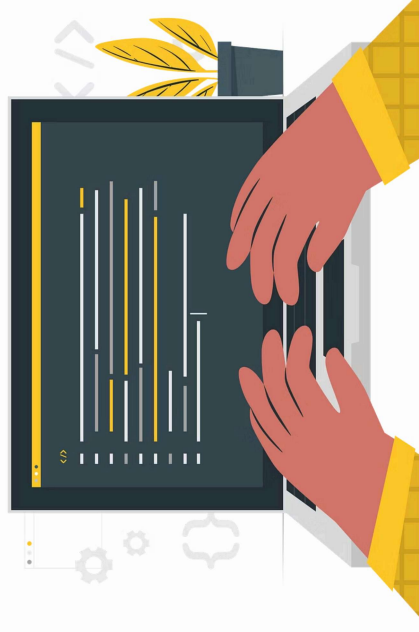
Eventos en JavaScript

Son cruciales para crear aplicaciones web interactivas y receptivas.

Son acciones que ocurren en el navegador y que podemos "escuchar" o capturar para ejecutar algún código. Podría definirse como una señal de que algo ha pasado en el navegador, y podemos responder a esa señal escribiendo código.

Interactividad

- Pueden ser disparados por distintas interacciones como un clic, una tecla presionada, o el envío de un formulario) o bien por el navegador (carga de la página) u otros elementos interactivos.



Tipos de Eventos Comunes



Click

Se dispara cuando se hace clic en un elemento.



Keypress

Se activa cuando se presiona una tecla.



Submit

Ocurre cuando un formulario es enviado.



Load

Ocurre cuando un recurso (como una página o una imagen) se ha cargado completamente.



Mouseover

- Se dispara cuando el ratón pasa por encima de un elemento.

Escuchando Eventos con addEventListener

Este método asocia una función a un evento para que se ejecute cada vez que el mismo ocurra.

- **Ejemplo:** cuando se haga clic en el botón, aparecerá una alerta. El método `addEventListener` nos permite definir de manera limpia cómo reaccionar a distintos eventos.

```
<button id="miBoton">Hacé clic</button>
<script>
let boton = document.getElementById("miBoton");
```

```
boton.addEventListener("click", function() {  
  alert("¡Hiciste clic en el botón!");  
});  
</script>
```

Diferencias entre Métodos de Manejo de Eventos

Atributos HTML

Directamente en HTML, no recomendado por mezclar estructura y lógica.

```
<button
  onclick="alert('¡Hiciste
  clic!')">Hacé
  clic</button>
```

Propiedades de elementos

Asignación directa de funciones a propiedades de eventos.

```
boton.onclick =
function() {
  alert("¡Clic
  detectado!");
};
```

addEventListener

Método recomendado, permite múltiples listeners y es más flexible.

Eventos con Funciones Flecha vs Tradicionales

Funciones tradicionales

'this' se refiere al elemento que disparó el evento.

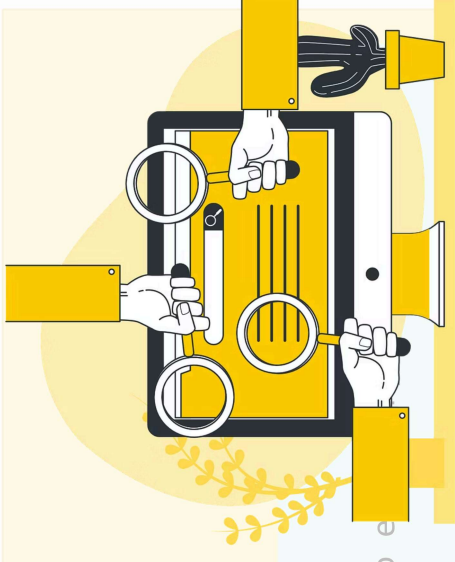
- Ejemplo:

```
boton.addEventListener("click", function() {  
    console.log(this); // 'this' se refiere al elemento que disparó el evento  
});
```

Delegación de Eventos

'this' se refiere al contexto exterior, no al elemento.

- Técnica que nos permite escuchar eventos de varios elementos hijos utilizando un solo "listener" en el padre.
- Es especialmente útil cuando los elementos hijos son generados dinámicamente.



Ejemplo:

En lugar de agregar un listener a cada `<li>`, lo añadimos a la lista completa y verificamos si el `target` del evento fue un `li`.

```
<ul id="lista">
  <li>Elemento 1</li>
  <li>Elemento 2</li>
  <li>Elemento 3</li>
</ul>
<script>
  let lista = document.getElementById("lista");
  lista.addEventListener("click", function(event) {
    if (event.target.tagName === "LI") {
      alert("Hiciste clic en " + event.target.textContent);
    }
  });
</script>
```

Ejercicios Prácticos

Ejercicio Práctico #1

Optativo | No
entregable

Modificación del DOM con Métodos de Selección

Pasos a seguir:

Crear un programa que, al hacer clic en un botón, cambie el contenido de un párrafo en la página utilizando los métodos para acceder al DOM.

1. Al cargar la página, se debe mostrar un párrafo con el texto: "Texto inicial".
2. Al hacer clic en un botón, se debe cambiar ese texto por: "El texto ha sido modificado con JavaScript".
3. Usar `getElementById()` para seleccionar el párrafo y modificar su contenido con `textContent`.

Tips claves:

- **Selección por id:** Utilizar `getElementById()` para acceder al elemento del DOM que queremos modificar.
- **Modificación de contenido:** Utilizá `textContent` para cambiar el texto del párrafo.
- **Eventos:** Usar `addEventListener` para ejecutar la modificación cuando se hace clic en el botón.



Ejercicio Práctico #2

Optativo | No
entregable

Pasos a seguir:

Desarrollar un programa que permita agregar elementos a una lista mediante un botón.

1. Al hacer clic en un botón, se debe crear un nuevo elemento `<li>` en una lista ya existente.
2. El contenido del nuevo elemento debe ser el texto: "Nuevo Elemento".
3. Usar `createElement()` para crear el nuevo elemento y `appendChild()` para añadirlo a lista.
4. Cada vez que se agrega un nuevo elemento, se debe mostrar una alerta indicando: "Se ha añadido un nuevo elemento".

Tips claves:

- **Creación de elementos:** Usar `createElement()` para generar el nuevo nodo `<li>`.
- **Agregar al DOM:** Usar `appendChild()` para añadir el nuevo elemento a la lista.
- **Eventos:** Utilizá `addEventListener()` para asociar la acción de agregar el elemento al evento click del botón.



¡Nuevo cuestionario en Campus!